

Lab 6: Polymorphism

Due **Sunday October 15th, 2023 at 8:00 PM ET** on Gradescope.

Each lab is graded for attendance (individual) and completion (group). **Note: Don't forget to add your pod members to the Gradescope submission!**

Getting started

1. One of your group members should create an editable copy of this worksheet to fill out and share the document with all other group members. (*File → Make a Copy* in the menu.)
2. Fill in the table below based on the letters given to each group member. If your group does not have exactly four members, please work out a strategy for sharing the roles that works for you.

	Name	Role (rotates each week)
A	Sonya Konon	Leader
B	Shikha Nukala	Scribe
C	Spence Lowery	Coder I
D		Coder II

1. The **scribe** should do a find/replace (select *Edit > Find and replace* from the menu) to change all instances of "Person A" to that member's name, "Person B" to that member's name, and so on.

Formatting

Please do not change the formatting of the document or which exercises appear on which slides. We use the placement of questions on each page to assist with grading.

Questions?

You are expected to lead the meeting yourselves and collaboratively work through the exercises, but please let us know if you have questions! We're here if you need us and will be checking in throughout the lab.

Note

These lab exercises were developed for in-person and virtual labs. Depending on the way you and your group are working, you are welcome to interpret the exercise instructions however works best (e.g. "share screen" means something different if you're in-person vs. virtual).

Answer the following questions about this class hierarchy:

```
class Base {
public:
    void print() { cout << "Base" << endl; }
private:
    int base_data;
};

class Derived : public Base {
public:
    void print() {
        cout << "Derived" << endl;
    }
    int addOne(int x) {
        return x + 1;
    }
    double addOne(double x) {
        return x + 1;
    }
private:
    int derived_data;
};
```

Question	Discussion Leader	Your answer
Given the classes defined above, which member functions are overloaded? Which are overridden?	Person C	Print is overridden
Base b; Base *b_ptr = &b; Derived *d_ptr = b_ptr; This code above will cause a compile error on the 3rd line. Imagine it did not cause a compiler error there - what is something bad it could do?	Person D	b_ptr is the memory address of the location of the Base b object. We put the memory address into the d_ptr variable. So when we try to declare the Derived pointer, there is no Base object it points to which would possible have undefined behavior.
Are either of the Base or Derived classes abstract? Are they interfaces?	Person A	Base is abstract, but derived is not. Base would be an interface.

Inheritance and Polymorphism Practice

Consider the following implementations to answer the questions on the next page.

```
class UofM {
public:
    UofM() { cout << "UofM ctor" << endl; }
    virtual ~UofM() {cout << "UofM dtor" << endl;}
    void print() { cout << "I'm a Wolverine!" << endl; };
};
```

```
class EECS : public UofM {
public:
    EECS() { cout << "EECS ctor" << endl; }
    virtual ~EECS() {cout << "EECS dtor" << endl;}
    void print() { cout << "I'm an EECS student!" << endl; }
    virtual void code() { cout << "I can code!" << endl; }
};
```

```
class EECS280 : public EECS {
public:
    EECS280() { cout << "EECS280 ctor" << endl; }
    virtual ~EECS280() {cout << "EECS280 dtor" << endl;}
    void print() { cout << "I'm an EECS280 student!" << endl; }
    void code() override { cout << "I like to code!" << endl; }
};
```

Question	Discussion Leader	Your answer
What does the following output? (Don't just run it through Lobster or your compiler!) <pre>int main() { EECS foo; foo.print(); EECS280 bar; bar.print(); bar.code(); return 0; }</pre>	Person B	UofM ctor EECS ctor Im an EECS student! UofM ctor EECS ctor EECS280 ctor Im an EECS280 student! I like to code!
What does the following output? <pre>int main() { EECS280 foo; EECS280 &bar = foo; EECS * ptr = &bar; ptr->print(); ptr->code(); return 0; }</pre>	Person C	UofM ctor EECS ctor EECS280 ctor Im an EECS student! I like to code!
What does the following output? <pre>int main() { EECS280 foo; UofM bar; UofM * ptr = &foo; ptr->print(); ptr = &bar; ptr->print(); return 0; }</pre>	Person D	UofM ctor EECS ctor EECS280 ctor UofM ctor Im a Wolverine! Im a Wolverine!

Subscribe to Restaurants with Polymorphism!

You've just started an internship at a software company that implements loyalty/rewards cards for restaurants. The `LoyaltyCard` class and its subclasses `NoThaiCard` and `PaneraCard` represent loyalty cards for various restaurants, but the benefits of each card work differently. At No Thai, you get every 10th meal free. At Panera, you get a 10% discount and free coffee.

The current implementation of a function `make_purchase()` uses an if/else structure to check which kind of card is being used. As more and more restaurants start to introduce loyalty cards for members, this code will become very complicated! The code also fails to compile in some cases, and you suspect this might have to do with misuse of inheritance/polymorphism.

```
class LoyaltyCard {
private:
    string type;
public:
    LoyaltyCard(const string &type_in) :
        type(type_in) { }

    string get_type() { return type; }

    // Assume this is implemented for you and
    // gives the regular price of the item
    double purchase(const string &item) {
        // Assume this is implemented
    }
};

class NoThaiCard : public LoyaltyCard {
private:
    int num_meals;
public:
    NoThaiCard() : LoyaltyCard("no_thai"),
        num_meals(0) { }

    int get_num_meals() { return num_meals; }
    void set_num_meals(int num_meals_in) {
        num_meals = num_meals_in;
    }
};

class PaneraCard : public LoyaltyCard {
public:
    PaneraCard() : LoyaltyCard("panera") { }
};
```

```
void make_purchase(double &money, LoyaltyCard
&card, const string &item) {

    if (card.get_type() == "no_thai") {
        if (card.get_num_meals() == 10) {
            card.set_num_meals(1);
            // no cost
        }
        else {
            card.set_num_meals(card.get_num_meals() +
1);
            money -= card.purchase(item);
        }
    }
    else if (card.get_type() == "panera") {
        if (item == "coffee") {
            // no cost
        }
        else {
            money -= 0.9 * card.purchase(item);
        }
    }
    else {
        // code should never get here
        assert(false);
    }
}

// *Disclaimer: this is a conceptual
representation; make_purchase() will not
actually compile due to attempts to call
derived class functions (e.g.
get_num_meals()) on a base class object
```

You know from taking EECS 280 that polymorphism and virtual functions can help clean up this code. Using the logic from the `make_purchase()` function without polymorphism (see previous page), fill in the code below for each derived class.

The `purchase()` function in `LoyaltyCard` should be virtual and each derived class should implement an override for that function. Once you've done this, you should be able to implement the external `make_purchase()` function in ONE line of code.

<pre>class LoyaltyCard { // Note that we no longer need a // string type; member variable public: // Assume this is implemented for you and // gives the regular price of the item virtual double purchase(const string &item) { // Assume this is implemented } };</pre>	<pre>void make_purchase(double &money, LoyaltyCard &card, const string &item) { // NEW implementation of // make_purchase(). This should // only be <i>*one*</i> line of code Money -= card.purchase(item); }</pre>
<pre>class NoThaiCard : public LoyaltyCard { private: int num_meals; public: NoThaiCard() : num_meals(0) { } double purchase(const string &item) override { if(card.get_num_meals() == 10{ card.set_num_meals(1); Return 0.0 } Return LoyaltyCard::purchase(item); } }</pre> <i>Hint: If your derived class <code>purchase()</code> function needs to call the version from the base class, you can do that using <code>LoyaltyCard::purchase()</code>.</i>	<pre>class PaneraCard : public LoyaltyCard { public: double purchase(const string &item) override { if(item == coffee){ Return 0.0; } Else{ Return .9 * LoyaltyCard::purchase(item); } } };</pre>

Pokemon Debugging

Let's return to the Pokemon mini-project from last time for some debugging practice.

We've created several buggy copies of our instructor solution. In this exercise, we'll walk you through some common debugging strategies as you find each bug.

The debugging strategies you practice here are covered in our [Debugging Guide](#) (available on [eecs280.org > Tutorials](#)) and will be helpful for debugging your Project 3 Euchre program.

Getting Started

You'll need a local clone of our project repository. *Note that this is a different repository from the previous lab*, so make sure to complete these steps. (You can't reuse the previous repository.)

Open a terminal and navigate to the folder where you keep your EECS 280 stuff. Then, use `git` to clone the starter files from our repository:

```
git clone https://github.com/jamesjuett/eecs280_pokemon_debugging.git
```

If you're using VS Code, you can open the folder with:

```
code eeecs280_pokemon_debugging
```

You're also welcome to use Visual Studio, Xcode, or any other visual debugger.

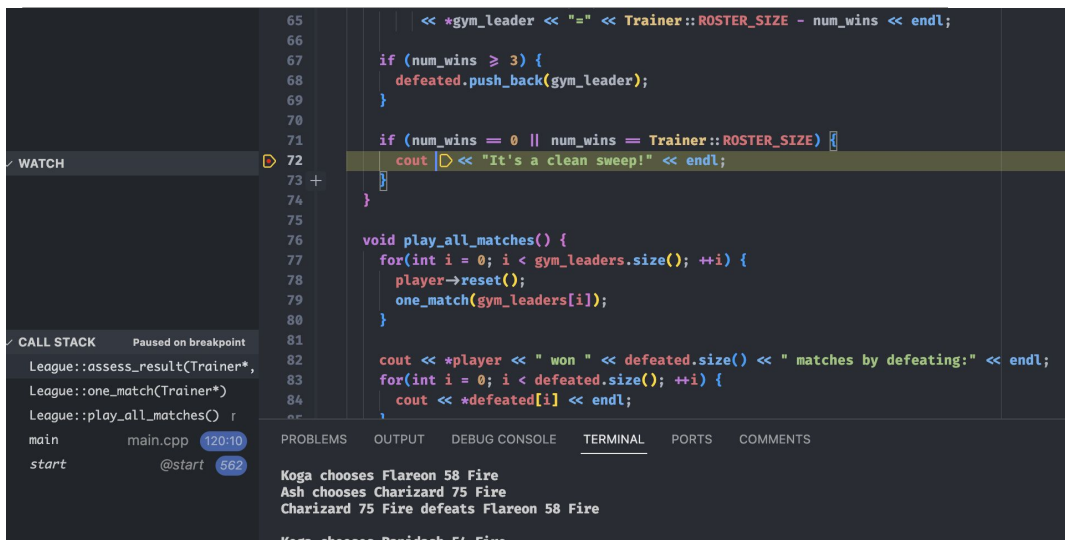
Debugger Setup

If you need a refresher, refer to the [setup tutorials](#) for your development environment. Or, ask a neighbor or your lab staff. A debugger is an essential tool for working with complex programs, and we want to make sure everyone is able to use one.

To confirm your setup:

1. Set a breakpoint on the line in `main.cpp` that prints "It's a clean sweep!".
2. If you're working in VS Code, remember to compile with `make main.exe`.
3. Configure your debugger. Remember you'll need to specify `ash.txt` and `indigo_league.txt` as **command line arguments**.
4. Run the program in the debugger. It should pause at your breakpoint.

Paste a screenshot of the debugger paused on your breakpoint into the box below.



Pokemon Debugging

Bug 1 - Assertion Failure, Explore the Call Stack

Make sure you have a terminal open in the base project directory. For example:

```
$ pwd
/home/jjuett/eecs280/eecs280_pokemon_debugging
```

Then, copy our first set of buggy files to your current directory with:

```
cp bug-1/* .
```

Make sure to recompile, then run the code. For example:

```
make main.exe
./main.exe ash.txt indigo_league.txt
```

You should see an assertion failure. Paste the error output into the box below.



```
Assertion failed: (!active_roster.empty()), function choose_pokemon, file Trainer.cpp, line 50.
```

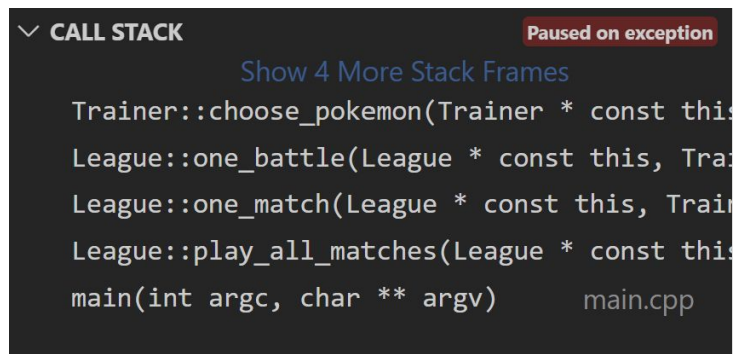
Look at the line number mentioned in the error, which is an assertion in the `choose_pokemon()` function. However, the actual bug is not in this function. How do we know that?

Not an issue in the function because the assert statement comes first in the function definition so function does not change anything in the `active_roster` in the implementation.

It appears our driver called `choose_pokemon()` too many times. To do that, run the code in your visual debugger (make sure to remove all breakpoints - we don't need any here).

Your debugger should pause on the failed assertion. It might show the assertion itself, or it might show some unhelpful system code.

Find where your debugger shows the **call stack**. A screenshot from VS Code is at the right. If your debugger wasn't showing the `choose_pokemon()` function, click it in the call stack so you can see that code.



You can see that `choose_pokemon()` was being called from `one_battle()`, which was being called from `one_match()`. Click the link in the call stack to inspect `one_match()`.

Find the **variables** display in your debugger. You should be able to see the value of the `i` variable - what is that value? Also, examine the loop in `one_match()`. What is the bug?

`i = 5`. The bug is that the code iterates from 0 to the roster size when iterating. It should iterate from 0 to one less than the size of the roster size so it does not go out of bounds.

Pokemon Debugging

Bug 2 - Output Mismatch, Set Breakpoint for More Info

Make sure you have a terminal open in the base project directory. For example:

```
$ pwd
/home/jjuett/eecs280/eecs280_pokemon_debugging
```

Then, copy a set of buggy files to your current directory with:

```
cp bug-2/* .
```

Make sure to recompile, then run the code. For example:

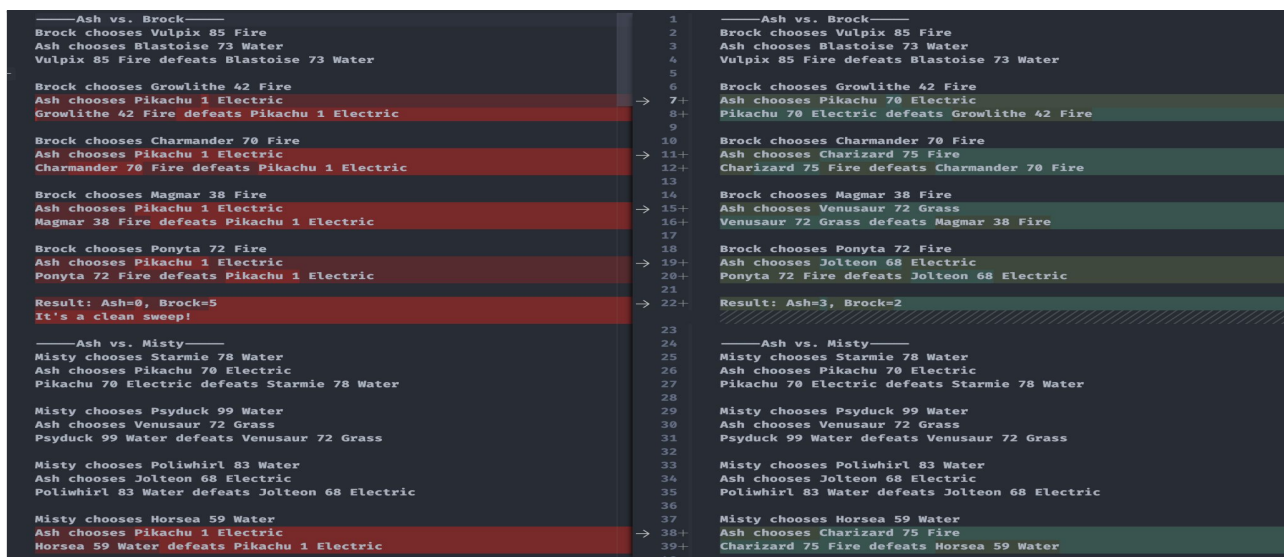
```
make main.exe
./main.exe ash.txt indigo_league.txt > main.out
```

Now, compare your main.out to main.out.correct.

You can use the integrated compare tool in VS Code. Right click on main.out and choose "Select for Compare". Then, right click main.out.correct and choose "Compare with Selected". You'll see the files side-by-side. (You may need to ensure your window is wide enough to display them correctly.)

Alternatively, you could copy+paste the contents of both files into an online diff tool or compare the files at the terminal with `sdiff main.out main.out.correct`.

Paste a screenshot of the side-by-side comparison in the box below.



(You may find this approach helpful for comparing your project 3 Euchre output as well.)

Why does Ash keep playing Pikachu?? We might form two hypotheses:

1. Ash wasn't initialized correctly, resulting in a team of mostly level 1 Pokachus.
2. Ash has the right pokemon, but the choose_pokemon() function is broken.

On the next page, we'll use the debugger to check the pokemon on Ash's team as the program runs, confirming or ruling out hypothesis #1.

Pokemon Debugging

Bug 2 - Continued

We'd like to check the pokemon on Ash's team while the program is running. Set a breakpoint in `main.cpp` on the first line of the `one_match()` function, shown at right.

```
47 void one_match(Trainer *gym_leader) {
48
49     cout << "-----" << *player << " vs. "
50
51     int num_wins = 0;
52     for(int i = 0; i < Trainer::POSTER_SIZE;
```

Next, run the debugger. Your previous configuration should work fine (i.e. run `main.exe` with `ash.txt` and `indigo_league.txt` as command line arguments.)

The debugger should pause at your breakpoint.

Find the **variables** display in your debugger.

Since `one_match()` is a member function, you should see a `this` pointer. You may need to click to expand it. This is where you can see **member variables**, like `player`.

Expand the `player` variable and check the elements of the `active_roster` vector. Paste a screenshot of these elements in the box at right.

```
/ name: Ash
> pokemon: size=5
v active_roster: size=0
v [raw]: std::_1::vector<Pokemon, std::_1::allocator<Pokemon>>
v std::_1::vector_base<Pokemon, std::_1::allocator<Pokemon>>: {__begin: 0x00000000100106b0, __end: 0x00000000100106b0}
v __begin: 0x00000000100106b0
  std::_1::vector_base<Pokemon, std::_1::allocator<Pokemon>>
  > name: std::_1::allocator<Pokemon>
    level: 68
    type: ELECTRIC
v __end: 0x00000000100106b0
  > name: "Jolteon"
    level: 68
    type: ELECTRIC
v __end: 0x00000000100106b0
```

Now, find your debugger controls and click the **"continue"** button **twice**. This will skip forward to the 3rd match Ash participates in. In the variables window, you can note that the `gym_leader` is now *Surge* (instead of Brock). Again, verify Ash still has the right pokemon for this match.

We can update our hypotheses:

1. ~~Ash wasn't initialized correctly, resulting in a team of mostly level 1 Pichachus.~~
2. Ash has the right pokemon, but the `choose_pokemon()` function is broken.

Don't exit your debugger yet. Add another breakpoint to the line shown at the right, where the player chooses a pokemon:

```
32 bool one_battle(Trainer *gym_leader) {
33     Pokemon enemy = gym_leader->choose_pokemon();
34     cout << *gym_leader << " chooses " << enemy << endl;
35     Pokemon p = player->choose_pokemon(enemy.get_type());
36     cout << *player << " chooses " << p << endl;
```

We want to check the behavior of this function. Click **"step into"**. Oops, it stepped into the `enemy.get_type()` function, which was on the same line. Just click **"step out"**, then **"step in"** again. Now you're in the `choose_pokemon()` function.

Now, use the "step over" (this might just be called "step", depending on your debugger) to walk through the operation of this function line-by-line. You'll eventually arrive at the final "return chosen;" line. In the box below, briefly describe why it (wrongly) contains a level 1 Pikachu.

Pikachu is initialized as a null Pokemon in the `choose_pokemon(adversary)` function. The problem is that we don't go to the `choose_pokemon()` function if there are no good pokemon that are against the adversary, so each time it won't return the best pokemon or the next in the active-roster it will return the null pokemon Pikachu.

Submit worksheet

Your pod is responsible for submitting lab worksheets to Gradescope by the due date listed on the first page. We strongly recommend submitting your worksheet on your lab day so you don't forget!

Download a PDF copy of your completed worksheet by going to *File > Download > PDF*.

The **Scribe** is responsible for submitting the PDF to Gradescope. **You need to add your group members to the submission after it is uploaded:**

